第21章 不相交集合：詳解

21-E01 Partition 與可重複 UNION

推導

初始6個單點。ab後{ab},{c},{d},{e},{f}共5；cd後{ab},{cd},{e},{f}共4；bc後{abcd},{e},{f}共3；重複ab維持3；ef後{abcd},{ef}共2；de後{abcdef}共1。成功合併5，UNION API為6，內部FIND為12。

外部m=6次MAKE-SET+6次UNION=12，不把內部FIND再加進外部m。若展開成MAKE/FIND/LINK序列，為6+12+5=23個（另有每次same-root比較常數成本）。相同根LINK會把未增加成員的rank升級；可對一個單點反覆UNION(x,x)，使rank任意大，違反集合至少2^r及rank≤log n。

結論與檢核

最多n−1的是成功合併，不是UNION API總呼叫數。根相同必跳過LINK；partition不變不代表任意rank更新仍合法。

21-E02 Naive 串列的真正成本

推導

Naive依序把原長串列接到新的單點後，改set人數1、2、3、4、5，共15，最後代表x_6。Weighted第1次同長保留x_2，之後新單點較短，改接到既有長串列；每次只改1人，共5，最後代表x_2。

一般Naive改寫$\Sigma_{i=1}^{n-1} i = n(n-1)/2$，另n次建立，時間$\Theta(n^2)$但API只有2n−1。Weighted在此序列改n−1人，時間Θ(n)。不過若成對合併同大集合，每輪仍需移n/2人，共$\log_2 n$輪，為Θ(n log n)，所以不可把此特殊序列的線性套到全部輸入。

結論與檢核

有tail只能把next串接成本降為O(1)，不能免除成員set反向指標更新。兩個算法都得到同一partition，但工作帳與代表不同。

21-E03 八元素 weighted list 的逐人成長

推導

改set次數依(a,b,c,d,e,f,g,h)為(0,1,1,2,1,2,2,3)，總12。h依次在UNION(g,h)後集合大小2、UNION(e,g)後4、UNION(a,e)後8，set依h→g→e→a。

$n=2^k$時第i輪有$n/2^i$次合併，每次移2^{i-1}人，乘積n/2，共k輪，所以總$(n/2)\log_2 n$。對任一x，若被移動，它原屬較小集合，合後大小至少兩倍；第j次後至少$2^j \le n$，因此$j \le \text{floor}(\log_2 n)$。這是逐元素計費，再加總所有元素。

第21章 不相交集合：詳解 （續）

結論與檢核

h達到三次上界，a零次；總和並非n $\log_2 n$=24，而是此平衡序列的12。上界不要求每個元素都取滿。

21-E04 同一組操作的 forest

推導

rank為(a,b,c,d,e,f,g,h)=(3,0,1,0,2,0,1,0)。parent為(a,a,a,c,a,e,e,g)，a自指。FIND(h)上行h→g→e→a，下行將h,g,e都接a，得到parent(a,a,a,c,a,e,a,a)，rank不變。

FIND(d)仍走d→c→a，之後才把d直接接a。若先覆寫parent[h]=a再以x=parent[h]繼續，立刻跳a，漏改g；e本就接a雖不影響值，仍未按照原路完成第二趟。

結論與檢核

需先old=parent[x]再覆寫，才保留下一個原祖先。Find不應把rank改為depth，因rank是歷史合併上界，且非根rank嚴格小於父rank的證明依賴它不被這樣重設。

21-E05 Rank 不是目前高度下界

推導

四個單點，UNION(a,b)得a.rank1，UNION(c,d)得c.rank1，再UNION(a,c)得a.rank2，d仍經c到a。FIND(d)將d接a，全部b,c,d皆直接接a，目前高度1但rank2。

對根rank作歸納：初始r0有$1=2^0$；只在兩個同rank r根合併時新根升r+1，兩集合互斥、各至少2^r，所以新集合至少2^{r+1}。不同rank不升，成員只增。非根rank是失去根時的值，當時曾需要至少2^r個元素，因此$2^r \le$最終n；不能把這個歷史下界寫成壓縮後該非根的目前subtree大小。

結論與檢核

$\lfloor \log_2 10^6 \rfloor = 19$；rank可能值0..19共20個，存值需$\lceil \log_2 20 \rceil = 5$bits。這不含parent索引與對齊開銷，並非整個節點只需5bits。

21-E06 level 與 iter 的實數值

推導

$A_0(j)=j+1$，所以迭代i次是j+i，$A_1(j)=2j+1$。對i歸納$A_1^{(i)}(j)=2^i(j+1)-1$，故$A_2(j)=2^{j+1}(j+1)-1$，$A_3(1)=A_2(A_2(1))=A_2(7)=2047$。迭代的下一步將前一步結果重新當輸入，不是乘上同一個數。

第21章 不相交集合：詳解 （續）

壓縮前g的r1、父rank2；e的r2、父rank3；p的r3、父rank4，三者level0、iter1。φ依序(3−0)・1−1=2、6−1=5、9−1=8。h.rank0故φ0，根r.rank4故φ12，這條path合計27。

壓縮後三者父rank皆4。g有$A_1(1)=3\le 4<A_2(1)=7$，故level1、iter1、φ1；e的$A_0(2)=3$、$A_0^2(2)=4$、$A_0^3(2)=5>4$，故level0、iter2、φ4；p仍level0、iter1、φ8。h與根不變，總25，減2。

結論與檢核

g,e後面都有同level非根，為好節點；p是該level最後非根，h為rank0起點，r為根，為例外。G靠level上升、e靠同level的iter上升，各令φ降1；iter不需要跨level也單調。

21-E07 位能不能只寫大 O 相消

推導

已知實際$\le 2s=14$，好節點2使$\Delta\Phi\le -2$，故$\Delta\Psi\le -6$，攤銷≤ 8。一般至少$g=\max(0,s-(\alpha+2))$，以$\Psi=C\Phi$、$C>c$，攤銷$\le cs-Cg$。$s\le\alpha+2$時至多$c(\alpha+2)$；$s>\alpha+2$時等於$(c-C)s+C(\alpha+2)\le C(\alpha+2)$，故$O(1+\alpha)$。固定C不能隨每次路徑臨時換值。

四次$\Delta\Psi=(4,-2,7,-3)$，攤銷為(7,5,9,5)，總26；實際總20。望遠鏡給$\Sigma a=\Sigma c+\Psi_{final}-\Psi_{initial}=20+6-0=26$，因此$\Sigma c=26-6=20\le 26$。

結論與檢核

有些操作會增加信用，有些取回；所需的是同一非負位能和整段前後差，不能把O(s)與$-\Omega(s-\alpha)$忽略係數直接相消。

21-E08 單次、總成本及小 n

推導

只rank時，$n=2^k$成對合併同rank根可造深度k；不壓縮則對同一深葉重複f次FIND需$\Theta(f\log n)$，先建樹另$O(n)$。任意LINK則可依次把舊根接新單點，形成長n−1的鏈，首次FIND $\Theta(n)$。同時rank與壓縮時，單次仍可能$\Theta(\log n)$，但整段m次$O(m(1+\alpha(n)))$；第一次長find的成本由整段位能與建立操作支付。

$\alpha(0)=0$、$\alpha(2)=0$、$\alpha(3)=1$、$\alpha(7)=2$、$\alpha(8)=3$、$\alpha(2047)=3$、$\alpha(2048)=4$。若n=1或2，MAKE/FIND仍有正常工作，不能寫$O(m\alpha)=0$，所以保留$1+\alpha$；$n\ge 3$才可簡寫$O(m\alpha)$。

結論與檢核

單次最壞與整段攤銷是不同分母。線性鏈反例沒有rank，不能用它宣稱rank+compression單次也可$\Theta(n)$。

第21章 不相交集合：詳解 （續）

21-E09 代表最小值與有向圖

推導

強制小鍵根做parent可能把高rank根接到低rank根，使parent-rank嚴格增加及高度分析失效。可保留rank根，在每個root存minKey：MAKE為自身鍵，LINK後新root.minKey=min(兩舊root.minKey)，加O(1)工作；查最小值先FIND再讀root.minKey，不要求結構根本身就是最小元素。

只有a→b時，忽略方向UNION會回同集合，但b不能到a，SCC應為{a},{b}。若無向圖只有ab邊，先合併a,b後刪ab，正確分量要拆成兩個；三操作只有建立單點與合併，沒有能復原舊分割的資料及操作。重新MAKE(a)亦違反元素只屬一集合的前提。

結論與檢核

可在root附加O(1)可合併的摘要，不任意更換連結規則。DSU適用只加入的無向連通，SCC或動態刪邊須另外演算法，不能從同代表推出有向互達。

21-E10 iter 重設與沿路 level 的反例

32 點：跨層的進度支付 iter 重設

$\alpha(32) = 3$。第一次 FIND(12) 將 rank 2 的節點 12 直接接到 rank 4 的根 0。UNION(16,0) 後，根 16 的 rank 升到 5，因此再次查找的路徑為 $12 \to 0 \to 16$，rank 為 $2, 4, 5$。

點	壓縮前 (ℓ, i, ϕ)	壓縮後 (ℓ, i, ϕ)
12	$(0, 2, 4)$	$(1, 1, 3)$
0	$(0, 1, 11)$	$(0, 1, 11)$
16	根，$\phi = 15$	根，$\phi = 15$

對 12，原父 rank 4 可容許兩次 A_0 迭代，但未達 $A_1(2) = 5$。新父 rank 5 達到第一層，只容許一次 A_1 迭代。位能差是 $-1 \cdot 2 + (2 - 1) = -1$；這次路徑位能由 30 降為 29，其餘節點不變，故全森林也是 $\Delta\Phi = -1$。12 後方有同 level 的非根 0，所以 12 是好節點；0 為該層最後一點，16 為根，均是例外。

第21章 不相交集合：詳解 （續）

16 點：不同節點的 level 可以下降

$\alpha(16) = 3$。類似建立後得到 $2 \to 0 \to 8$，rank 為 $1, 3, 4$。對節點 2，$A_1(1) = 3 \le 3 < A_2(1) = 7$，所以 $(\ell, i, \phi) = (1, 1, 1)$。對節點 0，$A_0(3) = 4 \le 4 < A_1(3) = 7$，所以 $(\ell, i, \phi) = (0, 1, 8)$。根 8 的位能為 12。
沿路 level 是 1、0，沒有同 level 的另一個非根，因此 $G = 0$。FIND(2) 後父 rank 由 3 變 4，但未跨過下一個門檻；三點的位能都不變。不同節點的比較不等於固定節點隨時間的比較，兩者並不衝突。

下界與實際個數

兩例皆為 $s = 3$、$\alpha = 3$，所以 $g = \max(0, 3 - 5) = 0$。第一例 $G = 1 \ge g$，第二例 $G = 0 = g$。短路徑可沒有好節點；$s \le \alpha + 2$ 時，直接以 $cs \le c(\alpha + 2)$ 支付成本，不需要位能下降。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第21章 不相交集合：詳解 （續）

原創驗算程式附錄: disjoint_depth_checks.py

```
"""Linked/forest partition checks and exact capped-Ackermann potential accounting."""
from functools import lru_cache
from random import Random
from math import log2


@lru_cache(None)
def ack(k,j,cap):
    if j>=cap:return cap
    if k==0:return min(cap,j+1)
    value=j
    for _ in range(j+1):
        value=ack(k-1,value,cap)
        if value>=cap:return cap
    return value


def alpha(n):
    k=0
    while ack(k,1,n+1)<n:k+=1
    return k


class Lists:
    def __init__(self,n):
        self.rep=list(range(n));self.members={x:[x] for x in range(n)}
        self.moves=[0]*n;self.sizes=[[] for _ in range(n)]
    def union(self,x,y,weighted=True):
        x=self.rep[x];y=self.rep[y]
        if x==y:return x
        if weighted and len(self.members[x])<len(self.members[y]):x,y=y,x
        old_size=len(self.members[y]);new_size=len(self.members[x])+old_size
        if weighted:assert new_size>=2*old_size
        for z in self.members[y]:
            self.rep[z]=x;self.moves[z]+=1;self.sizes[z].append(new_size)
        self.members[x]+=self.members.pop(y)
        return x
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第21章 不相交集合：詳解 （續）

```
class Forest:
    def __init__(self,n):
        self.p=list(range(n));self.rank=[0]*n;self.n=n;self.a=alpha(n)
        self.finds=self.good=self.links=0
    def path(self,x):
        out=[x]
        while self.p[x]!=x:x=self.p[x];out.append(x)
        assert len(out)<=self.n
        return out
    def parameters(self,x):
        r=self.rank[x];pr=self.rank[self.p[x]]
        if self.p[x]==x or r==0:return None
        k=0
        while ack(k+1,r,pr+1)<=pr:k+=1
        value=r;i=0
        while True:
            nxt=ack(k,value,pr+1)
            if nxt>pr:break
            value=nxt;i+=1
        assert 0<=k<self.a and 1<=i<=r
        return k,i
    def potential(self):
        total=0
        for x,r in enumerate(self.rank):
            parameter=self.parameters(x)
            value=self.a*r if parameter is None else (self.a-parameter[0])*r-parameter[1]
            assert value>=0;total+=value
        return total
    def find(self,x):
        path=self.path(x);before=self.potential()
        levels={z:self.parameters(z) for z in path[:-1] if self.rank[z]>0}
        good={z for j,z in enumerate(path[:-1]) if z in levels and any(y in levels and
levels[y][0]==levels[z][0] for y in path[j+1:-1])}
        assert len(good)>=max(0,len(path)-(self.a+2))
        root=path[-1]
        for z in path[:-1]:self.p[z]=root
        after=self.potential()
        assert after-before<=-len(good)
        assert 3*len(path)+4*(after-before)<=4*(self.a+2)
        self.finds+=1;self.good+=len(good)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第21章 不相交集合：詳解 （續）

```
        return root
    def union(self,x,y):
        x=self.find(x);y=self.find(y)
        if x==y:return x
        before=self.potential()
        if self.rank[x]<self.rank[y]:x,y=y,x
        self.p[y]=x
        if self.rank[x]==self.rank[y]:self.rank[x]+=1
        assert self.potential()-before<=self.a
        self.links+=1
        return x
    def check(self):
        groups={}
        for x in range(self.n):
            path=self.path(x);groups.setdefault(path[-1],[]).append(x)
            if self.p[x]!=x:assert self.rank[x]<self.rank[self.p[x]]
            assert len(path)-1<=self.rank[path[-1]]
        for root,members in groups.items():assert len(members)>=2**self.rank[root]
        return groups


def balanced(structure,n):
    size=1
    while size<n:
        for x in range(0,n,2*size):structure.union(x,x+size)
        size*=2


def main():
    rng=Random(21024);operations=0;good=0
    for n in [1,2,3,8,17,64]:
        f=Forest(n);linked=Lists(n);labels=list(range(n))
        for _ in range(600):
            x=rng.randrange(n);y=rng.randrange(n)
            if rng.random()<.55:
                old=labels[y];new=labels[x]
                labels=[new if z==old else z for z in labels]
                f.union(x,y);linked.union(x,y)
            else:f.find(x)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第21章 不相交集合：詳解 （續）

```
            roots=[f.path(i)[-1] for i in range(n)]
            assert all((roots[i]==roots[j])==(labels[i]==labels[j])==
(linked.rep[i]==linked.rep[j]) for i in range(n) for j in range(n))
            f.check();operations+=1
        assert max(linked.moves,default=0)<=int(log2(n))
        good+=f.good
    for n in [2,4,8,16,32,128]:
        l=Lists(n);balanced(l,n)
        assert sum(l.moves)==n*int(log2(n))//2
        assert max(l.moves)==int(log2(n))
        f=Forest(n);balanced(f,n);before=f.rank[:]
        assert f.path(n-1)==[n-1]+[n-2**i for i in range(1,int(log2(n))+1)]
        for x in range(n):f.find(x)
        assert f.rank==before and all(len(f.path(x))<=2 for x in range(n))
        f.check();good+=f.good
    for n in range(1,33):
        naive=Lists(n);weighted=Lists(n)
        for x in range(1,n):naive.union(x,x-1,False);weighted.union(x,x-1)
        assert sum(naive.moves)==n*(n-1)//2
        assert sum(weighted.moves)==max(0,n-1)
    f=Forest(8);balanced(f,8)
    assert f.path(7)==[7,6,4,0] and f.rank==[3,0,1,0,2,0,1,0]
    f.find(7);assert [f.p[z] for z in [7,6,4]]==[0,0,0]
    assert [alpha(n) for n in [0,1,2,3,4,7,8,2047,2048]]==[0,0,0,1,2,2,3,3,4]
    assert ack(1,3,100000)==7 and ack(2,3,100000)==63
    f=Forest(16);balanced(f,16);old=f.potential()
    assert f.path(15)==[15,14,12,8,0]
    assert [f.parameters(x) for x in [14,12,8]]==[(0,1),(0,1),(0,1)]
    f.find(15)
    assert [f.parameters(x) for x in [14,12,8]]==[(1,1),(0,2),(0,1)]
    assert old-f.potential()==2
    print(f'PASS mixed_partition_operations={operations}
list_and_forest_vs_explicit_equivalence=True '
          f'exact_potential_good_nodes={good} FIND_and_LINK_charge_inequalities=True '
          'balanced_moves_n8=12 naive_moves_n32=496 weighted_moves_n32=31 '
          'sample_h_g_e_a_ranks=0,1,2,3 compression_ranks_unchanged=True
alpha_small_bases=True')


if __name__=='__main__':main()
```

第21章 不相交集合：詳解 （續）

原創驗算程式附錄: union_rank_path_compression.py

```
"""Check the chosen inverse-Ackermann definition and path compression."""

from dataclasses import dataclass


def iterate(function, value, times):
    for _ in range(times):
        value = function(value)
    return value


def ackermann_level(j, level):
    if level == 0:
        return j + 1
    if level == 1:
        return 2 * j + 1
    if level == 2:
        return (1 << (j + 1)) * (j + 1) - 1

    def previous(value):
        return ackermann_level(value, level - 1)

    return iterate(previous, j, j + 1)


def alpha_through_three(n):
    level = 0
    while level <= 3 and ackermann_level(1, level) < n:
        level += 1
    assert level <= 3
    return level


@dataclass
class Node:
    name: str
    parent: object = None
    rank: int = 0

    def __post_init__(self):
        if self.parent is None:
            self.parent = self


def find(node):
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第21章 不相交集合：詳解 （續）

```
    path = []
    while node.parent is not node:
        path.append(node)
        node = node.parent
    for item in path:
        item.parent = node
    return node


def union(a, b):
    root_a, root_b = find(a), find(b)
    if root_a is root_b:
        return root_a
    if root_a.rank > root_b.rank:
        root_a, root_b = root_b, root_a
    root_a.parent = root_b
    if root_a.rank == root_b.rank:
        root_b.rank += 1
    return root_b


def run():
    expected = {2: 0, 3: 1, 7: 2, 8: 3, 2047: 3}
    for n, level in expected.items():
        assert alpha_through_three(n) == level, (n, alpha_through_three(n), level)
    assert ackermann_level(1, 3) == 2047
    assert ackermann_level(1, 3) < 2048
    # A4(1)=A3(A3(1)); A3 is strictly increasing and A3(1)=2047,
    # so the next value is at least 2048. The huge value is not materialized.
    assert 2048 == ackermann_level(1, 3) + 1

    root, p, x = Node("root", rank=2), Node("p", rank=1), Node("x", rank=0)
    p.parent = root
    x.parent = p
    assert find(x) is root
    assert x.parent is root and p.parent is root
    assert (x.rank, p.rank, root.rank) == (0, 1, 2)
    print("PASS ackermann cases=5 alpha2048=4_proved path_compressed=2 ranks_unchanged=3")
    return expected


if __name__ == "__main__":
    run()
```

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study